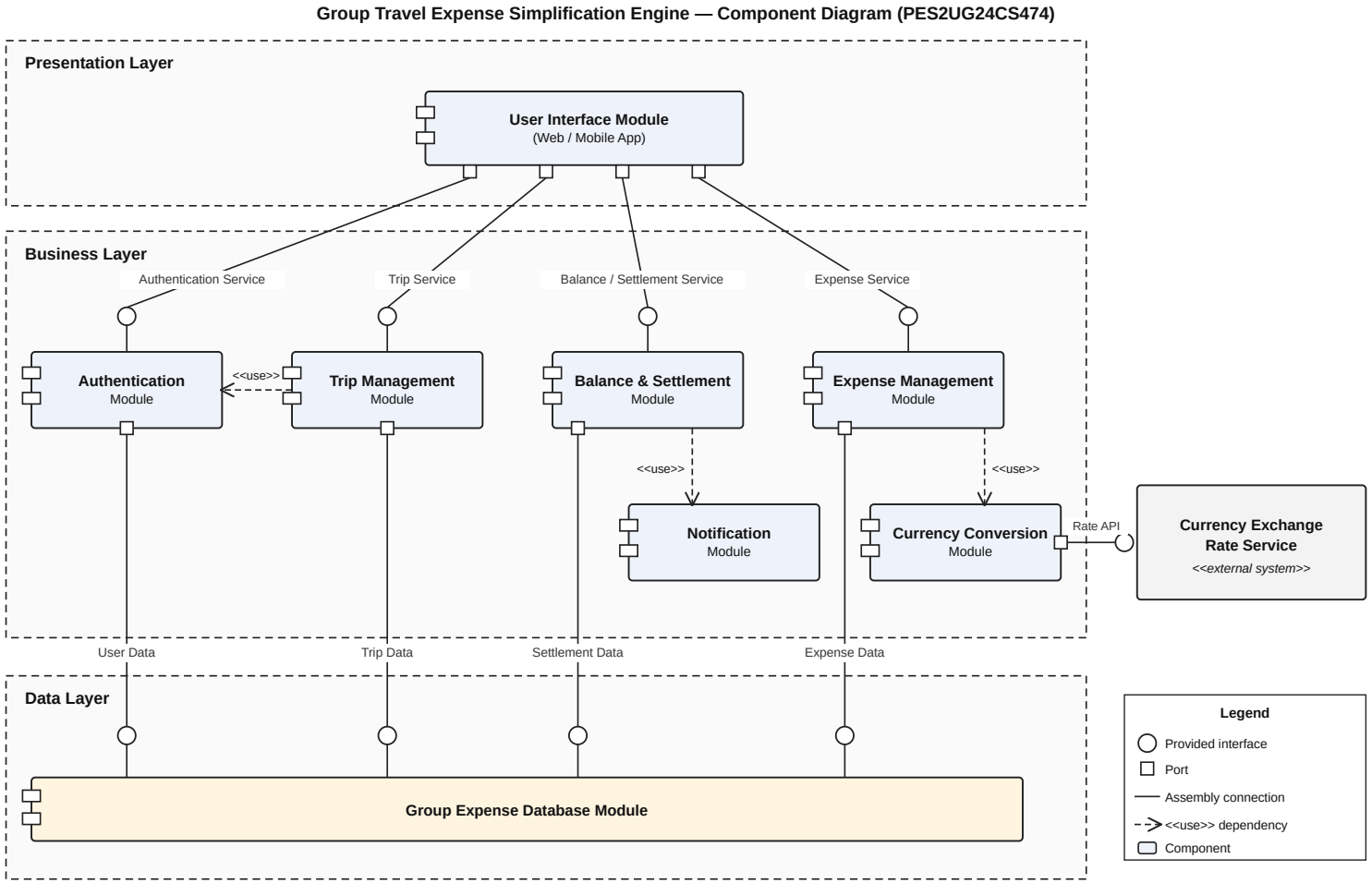
Lab 3 SE

Group Travel Expense Simplification Engine – Architectural Justification

NAME: SHIVANSH SHARMA SRN: PES2UG24CS474 SECTION: H

COMPONENT DIAGRAM



Architecture Selection

We chose a **Layered Architecture** for the Group Travel Expense Simplification Engine (Problem Statement #37). The system is divided into three layers: the Presentation Layer, the Business Layer and the Data Layer. Each layer only talks to the layer directly below it, so the user interface never reaches the database directly.

Architectural Choice

The Presentation Layer holds the User Interface Module, through which Group Travelers and the Trip Admin log expenses, view balances and trigger “Settle Up”. The Business Layer holds the Authentication Module (UC-07), Trip Management Module (UC-05), Expense Management Module (UC-01), Currency Conversion Module (UC-02), Balance & Settlement Module (UC-03, UC-04) and Notification Module (UC-06). The Data Layer holds the Group Expense Database Module, which stores users, trips, expenses and settlements. The only component outside the layers is the external Currency Exchange Rate Service, reached through the Exchange Rate API required by the Currency Conversion Module.

Component	Use cases	Requirements
Authentication Module	UC-07	NFR-002 (secure access)
Trip Management Module	UC-05	FR-004
Expense Management Module	UC-01	FR-002
Currency Conversion Module	UC-02	FR-003
Balance & Settlement Module	UC-03, UC-04	FR-001, FR-005, NFR-001
Notification Module	UC-06	Settlement and payment reminders
Group Expense Database Module	All	NFR-002 (AES-256 at rest)

Reason 1 – Separation of Responsibilities

The system has very different kinds of work: showing screens, splitting and converting expenses, running the min-flow settlement algorithm, and storing data. A layered design keeps each of these in its own place. For example, the split-validation rule of FR-002 (shares must add up to the total) lives only in the Expense Management Module, and the optimal-settlement logic of FR-001 lives only in the Balance & Settlement Module. The UI simply calls these services. This makes the system easier to understand, test and debug, since a bug in a settlement result can be traced to one module rather than spread across the UI and database code.

Reason 2 – Maintainability and Modularity

Modules talk to each other only through defined interfaces, such as the Expense Service, Trip Service and Settlement Data interfaces shown in the diagram. The Currency Conversion Module is the clearest example. Expense Management depends on it through a <<**use**>> dependency (UC-01 includes UC-02), and only this module knows how to talk to the Exchange Rate API. If the exchange-rate provider is changed, only this module needs to change, and expense logging, balances and settlement are unaffected. In the same way, the database can be upgraded or replaced without touching the business modules, because they depend only on the Data Layer interfaces.

Security Advantage

NFR-002 requires TLS 1.2+ for data in transit and AES-256 for data at rest. In this architecture, all client traffic enters at a single point, the User Interface Module, where TLS can be enforced. Encryption at rest is applied inside the Group Expense Database Module, so no business module has to handle encryption itself, and no raw payment information is stored. Authentication is a separate module: it verifies users at login, and the Trip Management Module uses it (<<**use**>>) to confirm Trip Admin rights before travelers are added or removed. Because the UI cannot reach the database directly, an unauthenticated user cannot bypass the business rules to read or change financial data.

Performance Benefit

NFR-001 requires settlement to run in under 100 ms for 50 travelers and 500 expenses. The Balance & Settlement Module reads already-converted expenses through the Settlement Data and Expense Data interfaces, computes net balances and runs the min-flow algorithm in memory, so the calculation is not slowed down by UI code or by currency lookups, which are completed earlier by the Currency Conversion Module (UC-02 is a precondition of UC-04). Since the Data Layer centralises database access, queries can be optimised and indexed in one place, and live balance updates for FR-005 only need to read the data they require.

Conclusion

The Layered Architecture gives the Group Travel Expense Simplification Engine a clear and organised structure. Its use of components, ports, provided interfaces, assembly connections and <<use>> dependencies shows how presentation, business logic, data storage and the external exchange-rate service interact, while keeping the system maintainable, secure, and fast enough to meet the requirements from Lab 1.